

Case Study : Smart Logistics and Supply Chain Analytics System using Apache Spark

A logistics company collects shipment records, warehouse data, GPS tracking information, fuel consumption records, and customer delivery data. The company wants to develop a **Supply Chain Analytics Platform** to optimize logistics operations and improve delivery efficiency.

As a Big Data Engineer, design, implement, and deploy the solution using Apache Spark and PySpark.

Q1. Spark Initialization and Data Loading (2 Marks)

Initialize Spark and load logistics datasets.

Q2. RDD Implementation (3 Marks)

Create RDDs and perform transformation and action operations.

Q3. Key-Value Operations and Persistence (2 Marks)

Implement key-value operations, shuffle operations, and persistence.

Q4. Spark DataFrame Operations (3 Marks)

Perform DataFrame operations including filtering, grouping, joins, and aggregation.

Q5. Exploratory Data Analysis and Spark SQL (5 Marks)

Perform EDA and execute Spark SQL queries to:

- Analyze delivery performance.
- Determine warehouse utilization.
- Identify delayed shipments.
- Analyze fuel consumption trends.
- Generate regional logistics reports.

Q6. ETL Pipeline Development (3 Marks)

Develop ETL pipelines for shipment, warehouse, and tracking data.

Q7. Machine Learning/Deep Learning Implementation (2 Marks)

Implement a model to predict delivery delays or optimize logistics routes.

Additional Implementation Instructions

A. Version Control using Git and GitHub

- Create a **GitHub repository** for the project.
- Upload all source code, datasets (or dataset links), notebooks, and documentation.
- Maintain proper folder structure and commit history.
- Include a **README.md** file containing project description, installation steps, execution procedure, and output screenshots.

B. Containerization using Docker

- Create a **Dockerfile** for the Spark application.
- Build and test the Docker image.

C. Deployment using Kubernetes

- Create Kubernetes deployment and service YAML files.
- Deploy the application on a Kubernetes cluster (Minikube/Kubernetes/OpenShift).
- Verify successful deployment.

D. DevOps Pipeline Implementation

- Configure a basic CI/CD pipeline using GitHub Actions/Jenkins/GitLab CI.
- Automate build, testing, Docker image creation, and deployment.

E. Project Submission Requirements

Submit:

- Source code
- GitHub repository link
- Dockerfile
- Kubernetes YAML files
- CI/CD pipeline configuration
- Execution screenshots
- EDA and Spark SQL outputs
- Model evaluation report
- Final project documentation